AI used: Claude Sonnet 5 model
First prompt: (9/27/2026 2:40 am)

C++ program that reads a line of text and extracts a single valid IPv4 address, followed by a port number. Only digits, periods and colons are cared about ignore all other characters. It must fully match all rules with no exceptions.

An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional `:port` may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

C++

```
// Returns true if a valid address was found, false otherwise.

// On success: outAddress holds the 32-bit value,

// and outPort holds the port number, or -1 if no port was present.

// On failure: outAddress is set to 0 and outPort is set to -1.

bool extractIPv4(const std::string& str, unsigned long& outAddress,
int& outPort);
```

Do not use at any string-to-number conversion function such as: atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf with numeric conversions. DON'T USE ANY OF THESE. Same with any address-parsing library function: `inet_aton, inet_pton, inet_addr,` or equivalents. Any regular-expression facility (std::regex, POSIX regex.h, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.

- Only one valid address per input, ignore anything after finding the first address or set it to fail.
- Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.
- On success, print: `Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)` where N is the 32-bit decimal value and P is the port number or the literal text none.
- Loop until userinput is "END" then end the program.
- Output to terminal

Must be able to handle everysingle case below if not more edge cases not prompted.

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now

Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:
none)

Enter a string (or 'END' to quit): server=10.0.0.255:8080end

Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port:
8080)

Enter a string (or 'END' to quit): 192a168.1.1.1

Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port:
none)

Enter a string (or 'END' to quit): 192.168.1.1.

Invalid input: no valid IPv4 address found

Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused

Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:
none)

Enter a string (or 'END' to quit): 192.168.01.1

Invalid input: no valid IPv4 address found

Enter a string (or 'END' to quit): 1.2.3.4:99999

Invalid input: no valid IPv4 address found

Enter a string (or 'END' to quit): 12.34.56

Invalid input: no valid IPv4 address found

Enter a string (or 'END' to quit): no number here

Invalid input: no valid IPv4 address found

Enter a string (or 'END' to quit): END

Program terminated.
```

AI Output:

Here's a complete C++ program that does character-by-character parsing with no library conversion/regex/inet functions — it tokenizes runs of `[0-9.:]` characters (treating anything else as a plain separator) and then strictly validates each run against the full grammar.

```
#include <iostream>
#include <string>

// ---------- Helpers ----------

static bool isRelevant(char c) {
    return (c >= '0' && c <= '9') || c == '.' || c == ':';
}

static bool isDigitChar(char c) {
    return c >= '0' && c <= '9';
}

// Reads a maximal run of digit characters starting at pos (within [start,end)).
// Sets digitsEnd to one-past-the-last digit. Returns false if there are no digits at
pos.
static bool readDigits(const std::string& s, size_t pos, size_t end, size_t&
digitsEnd) {
    size_t p = pos;
    while (p < end && isDigitChar(s[p])) {
        ++p;
    }
    if (p == pos) return false; // empty digit run
    digitsEnd = p;
    return true;
}

// Validates the digit substring [start,end) as a numeric field:
//  - length must be 1..maxLen
//  - no leading zero unless the field is the single character '0'
//  - numeric value must be <= maxValue
// On success, sets value.
static bool validateField(const std::string& s, size_t start, size_t end,
                          size_t maxLen, long maxValue, long& value) {
    size_t len = end - start;
    if (len == 0 || len > maxLen) return false;
    if (len > 1 && s[start] == '0') return false; // disallowed leading zero

    long val = 0;
    for (size_t i = start; i < end; ++i) {
        val = val * 10 + (s[i] - '0');
    }
    if (val > maxValue) return false;

    value = val;
    return true;
}

// Attempts to parse the ENTIRE run [start,end) as: octet.octet.octet.octet(:port)?
```

```
// Every character in the run must be consumed by the grammar, or it fails.
static bool parseAddressToken(const std::string& s, size_t start, size_t end,
                              unsigned long& addr, int& port) {
    size_t pos = start;
    long octets[4];

    for (int i = 0; i < 4; ++i) {
        size_t digitsEnd;
        if (!readDigits(s, pos, end, digitsEnd)) return false;

        long val;
        if (!validateField(s, pos, digitsEnd, 3, 255, val)) return false;

        octets[i] = val;
        pos = digitsEnd;

        if (i < 3) {
            if (pos >= end || s[pos] != '.') return false; // must be separated by '.'
            ++pos; // consume '.'
        }
    }

    // After the 4th octet: either end of run (no port), or ':' + valid port, and
    // nothing may be left over afterward.
    if (pos == end) {
        port = -1;
    } else if (s[pos] == ':') {
        ++pos;
        size_t digitsEnd;
        if (!readDigits(s, pos, end, digitsEnd)) return false;

        long val;
        if (!validateField(s, pos, digitsEnd, 5, 65535, val)) return false;

        port = static_cast<int>(val);
        pos = digitsEnd;
        if (pos != end) return false; // trailing junk after the port
    } else {
        return false; // stray '.' or other leftover right after the 4th octet
    }

    addr = (static_cast<unsigned long>(octets[0]) << 24) |
           (static_cast<unsigned long>(octets[1]) << 16) |
           (static_cast<unsigned long>(octets[2]) << 8)  |
           (static_cast<unsigned long>(octets[3]));
    return true;
}

// ---------- Public API ----------

// Returns true if a valid address was found, false otherwise.
// On success: outAddress holds the 32-bit value,
// and outPort holds the port number, or -1 if no port was present.
// On failure: outAddress is set to 0 and outPort is set to -1.
bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort) {
```

```
    size_t n = str.size();
    size_t i = 0;

    while (i < n) {
        if (!isRelevant(str[i])) {
            ++i;
            continue;
        }

        size_t runStart = i;
        while (i < n && isRelevant(str[i])) {
            ++i;
        }
        size_t runEnd = i; // run is [runStart, runEnd)

        unsigned long addr;
        int port;
        if (parseAddressToken(str, runStart, runEnd, addr, port)) {
            outAddress = addr;
            outPort = port;
            return true;
        }
        // otherwise keep scanning for the next run
    }

    outAddress = 0;
    outPort = -1;
    return false;
}

// ---------- Driver ----------

int main() {
    std::string line;

    while (true) {
        std::cout << "Enter a string (or 'END' to quit): ";
        if (!std::getline(std::cin, line)) {
            break;
        }

        if (line == "END") {
            std::cout << "Program terminated." << std::endl;
            break;
        }

        unsigned long addr;
        int port;

        if (extractIPv4(line, addr, port)) {
            unsigned int a = (addr >> 24) & 0xFF;
            unsigned int b = (addr >> 16) & 0xFF;
            unsigned int c = (addr >> 8) & 0xFF;
            unsigned int d = addr & 0xFF;
```

```
            std::cout << "Extracted IPv4 address: "
                      << a << "." << b << "." << c << "." << d
                      << " (decimal value: " << addr << ", port: ";
            if (port == -1) {
                std::cout << "none";
            } else {
                std::cout << port;
            }
            std::cout << ")" << std::endl;
        } else {
            std::cout << "Invalid input: no valid IPv4 address found" << std::endl;
        }
    }

    return 0;
}
```

First Run:

Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port: 8080)
Enter a string (or 'END' to quit): 192a168.1.1.1
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port: none)
Enter a string (or 'END' to quit): 192.168.1.1.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): 192.168.01.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated.
PS C:\EECS581\Assignment1>

The AI handles all given test Cases

Test Cases Written By Me:
Input: word
Output: Invalid input: no valid IPv4 address found

Input: 127.0.0.1 : 0
Output: Extracted IPv4 address: 127.0.0.1 (decimal value: 2130706433, port: none)

Input: 127.0.0.1 :0
Output: Extracted IPv4 address: 127.0.0.1 (decimal value: 2130706433, port: none)

Input: 127.0.0.1:0
Output: Extracted IPv4 address: 127.0.0.1 (decimal value: 2130706433, port: 0)

Input: 23END2.49.29.1:0
Output: Extracted IPv4 address: 2.49.29.1 (decimal value: 36773121, port: 0)

Input: 23END2.49.29.1:65536
output: Invalid input: no valid IPv4 address found

Input: 23END2.49.29.1:65535
Output: Extracted IPv4 address: 2.49.29.1 (decimal value: 36773121, port: 65535)

Input: 1.2.3.4:
Output: Invalid input: no valid IPv4 address found

Input: 1.2.3.4::5
Output: Invalid input: no valid IPv4 address found

Input: 1234.1.1.1
Output: Invalid input: no valid IPv4 address found

Input: 12.32.41.1:93 12.42.31.24:92
Output: Extracted IPv4 address: 12.32.41.1 (decimal value: 203434241, port: 93)

Verification statement:
The code for this assignment is 100% AI written, . Using claude I've noticed its ability to generate well performing code is far above any other language I've used so far. The reason it was able to get such an accurate program was due to the prompt given to it and the prompts accuracy to the requirements of the assignment.I've gone through and understand the operations and design of the c++ program, created by ClaudAI. I've gone through given test cases and edge cases I could come up with and the program passes each one I've prompted. I've ran through edge cases, structural order and other cases to get the wrong output but each time has worked so far. User UI is setup properly and how it'd intended from the assignment. The program works by taking in user input string, parses it into a token. It then using the grammar rules described and implemented earlier. Goes through the values of the token

matching and consuming values until either the entire token is eaten or it raises invalid input or parsing error. Through the use of multiple helper functions, to extract, match, and parse out the octets and port number. Comments added that are mine are hte ones that have double double forward slash (// //) to help explain code and to show understanding of the undercommented sections. Each helper class is well explained with the given comment the first couple lines of each function.

Limitations in code and understanding:
I'm unsure if I'm suppose to ignore a space in the text or not, since it would technically break the octet rule. Unsure though fully, so figured better to mention it, than not. My program currently only can see the port if there is no space between the IPv4 and port's colon and if colon is left blank afterwards it returns invalid input. I'm not sure how I should have the program handle this, so I'm going to leave it as I have it now. Even if stated obviously in the assignment instructions, I'm still unsure. I was also unable to get the program to fail, I know the assignment says to not trust a first time success, but I genuinely can't find anything wrong with the program. So anything wrong with it would be above what I'm capable off.